

Arquitetura, Implantacao e Automacao de Software

Projeto REVIVE - API Node.js/Express, Painel React/Vite e Supabase

Aluno: Vitor de Aguiar Adriaio

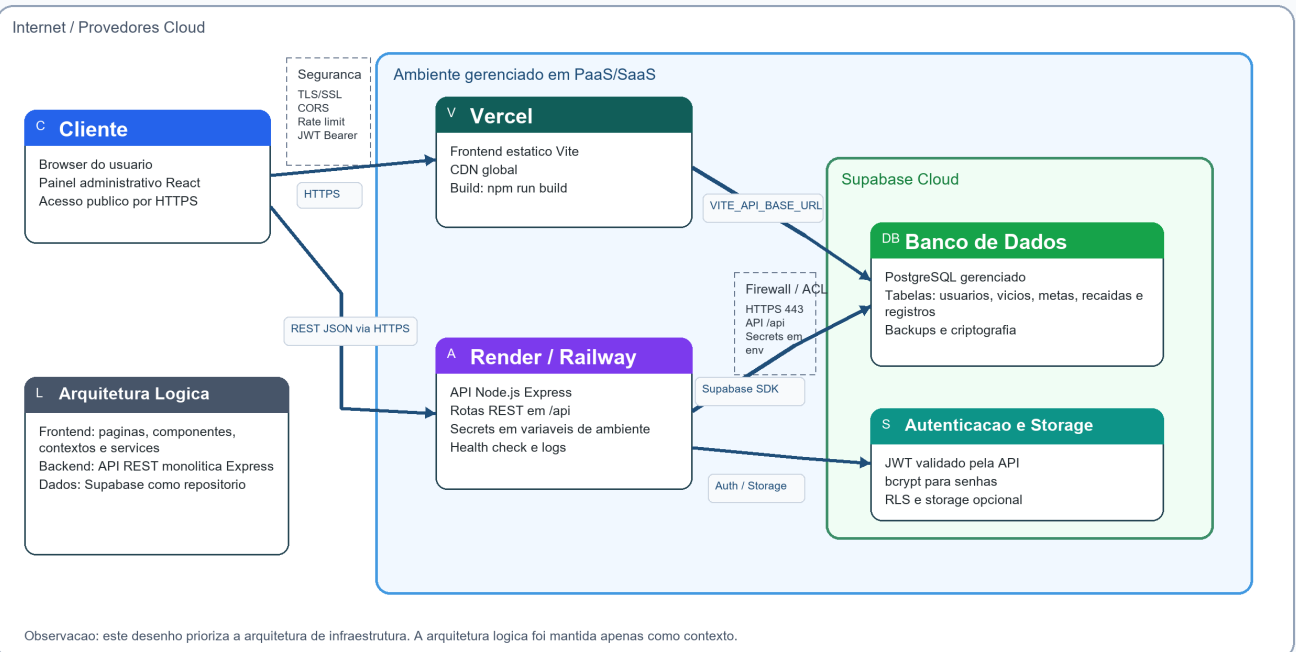
Matricula: 2310796

1. Diagrama de Arquitetura e Deploy

O diagrama abaixo representa a arquitetura de infraestrutura do REVIVE. A camada de cliente acessa o painel pelo navegador, o frontend e servido como aplicacao estatica em CDN, a API roda em PaaS Node.js e a camada de dados utiliza Supabase Cloud com PostgreSQL gerenciado. A seguranca aparece por HTTPS/TLS, CORS, rate limiting, JWT, variaveis de ambiente e controles do Supabase.

REVIVE - Arquitetura de Infraestrutura e Deploy

Foco em onde o codigo roda, como os servicos se comunicam e quais controles de seguranca protegem a aplicacao.
Aluno: Vitor de Aguiar Adriaio - Matricula: 2310796

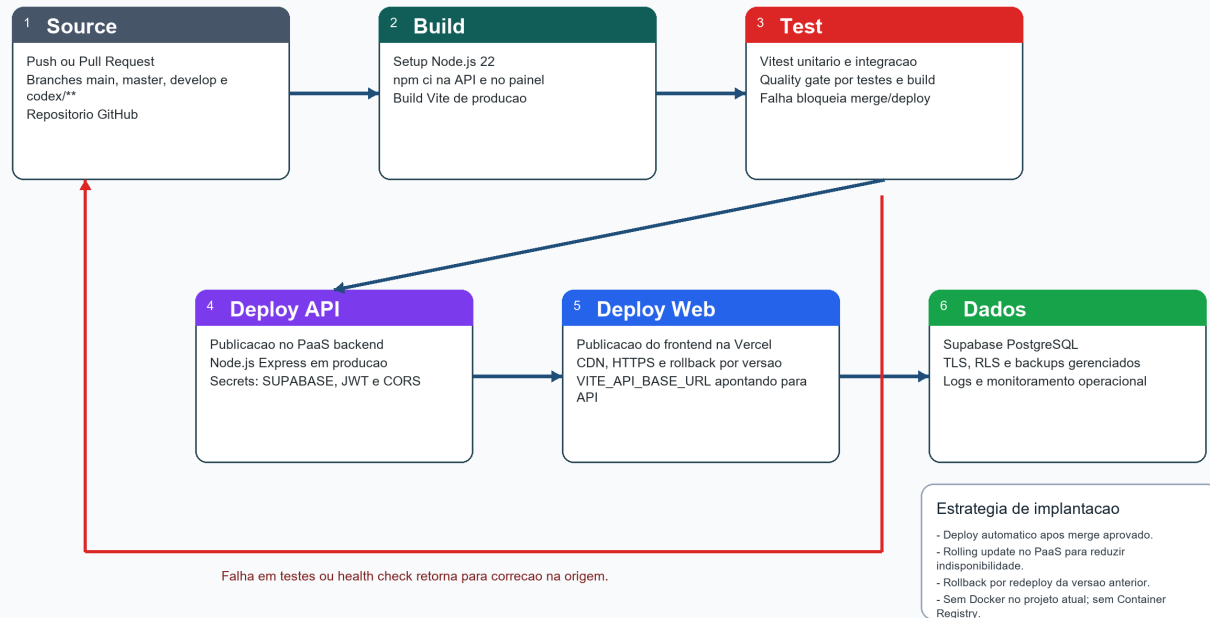


2. Pipeline de CI/CD

O fluxo automatizado usa GitHub como origem e GitHub Actions para validar a aplicacao. Como o projeto atual nao possui Dockerfile nem docker-compose, a pipeline nao utiliza Container Registry; os artefatos principais sao o build estatico do painel e o pacote Node.js da API.

REVIVE - Pipeline CI/CD

Fluxo automatizado do codigo ate producao com GitHub Actions, Vercel, PaaS backend e Supabase.



3. Relatório Técnico de Implantação

A. Estratégia de Deploy

A infraestrutura proposta combina PaaS e SaaS para reduzir a complexidade operacional. O frontend React/Vite é implantado em uma plataforma como Vercel, que entrega arquivos estáticos por CDN, aplica HTTPS automaticamente e permite rollbacks por versão. A API Node.js/Express é executada em um PaaS como Render ou Railway, mantendo o processo Node em ambiente gerenciado, com logs, health checks e variáveis de ambiente protegidas. O banco de dados fica no Supabase, que oferece PostgreSQL gerenciado, conexão TLS, backups, políticas de acesso e painel de administração.

Essa escolha é adequada ao REVIVE porque o projeto não exige controle de baixo nível de máquinas virtuais. O uso de PaaS/SaaS melhora a disponibilidade ao delegar balanceamento, renovação de certificados, reinício de processos, provisionamento e manutenção da base de dados ao provedor. Em produção, a disponibilidade é reforçada por CDN no frontend, health check no backend, configuração de variáveis de ambiente, backups do banco e separação clara entre apresentação, API e dados.

A segurança da implantação usa HTTPS em todos os acessos externos, CORS limitado às origens do painel, JWT Bearer para rotas autenticadas, bcrypt para armazenamento de senhas, rate limiting nas rotas da API e secrets fora do repositório. O Supabase complementa a proteção com conexão segura, controles de acesso e backup gerenciado.

B. Fases da Pipeline de CI/CD

Fase	Descrição alinhada ao projeto REVIVE
Source	O gatilho ocorre em push para main, master, develop ou codex/** e também em pull requests, conforme .github/workflows/tests.yml. O código fonte é versionado no GitHub e passa por revisão antes de chegar à branch principal.
Build	O GitHub Actions provisiona Node.js 22, executa npm ci na API e no painel, e cria o build de produção do frontend com npm run build:web. No estado atual não há imagem Docker; portanto não há publicação em registry.
Test	A pipeline executa testes unitários e de integração da API com Vitest, testes unitários e de integração do painel, e usa o build do Vite como etapa de validação. Falhas bloqueiam merge e deploy.
Deploy	Após merge aprovado, o frontend é publicado na Vercel e a API é publicada no PaaS backend. A estratégia indicada é rolling update do PaaS, com rollback por redeploy da versão anterior caso health checks ou validações falhem.
Operação	Logs do PaaS, painel do Supabase, backups e health checks acompanham a aplicação em produção. As variáveis SUPABASE_URL, SUPABASE_KEY, JWT_SECRET, ALLOWED_ORIGINS e VITE_API_BASE_URL ficam configuradas como secrets nos provedores.

Conclusão

A arquitetura proposta separa responsabilidade de cliente, aplicação e dados, mantendo coerência com o código existente: React/Vite no painel, Express na API e Supabase como persistência. A pipeline automatizada valida dependências, testes e build antes da implantação, reduzindo risco de regressão e tornando o caminho até produção repetível.